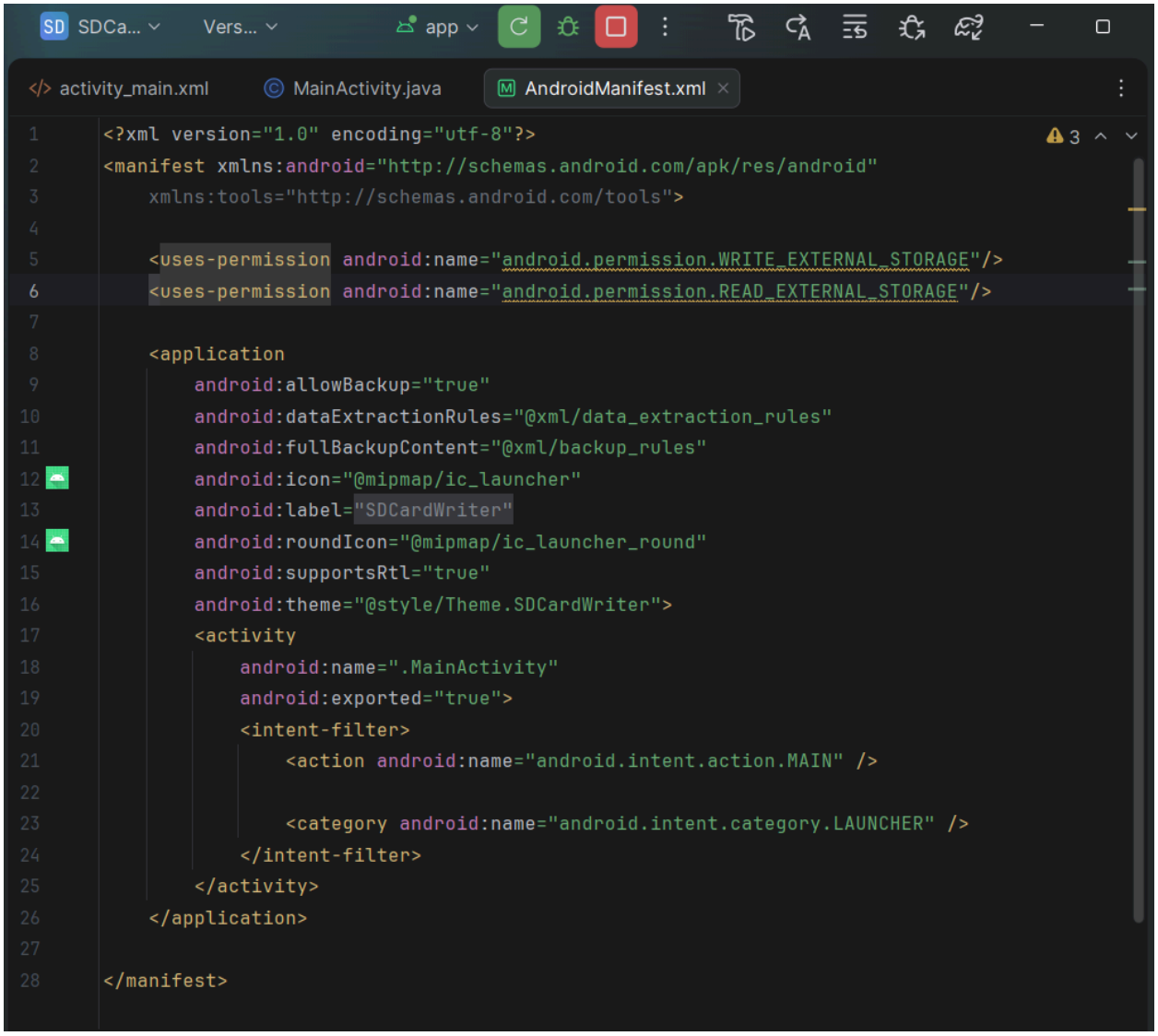
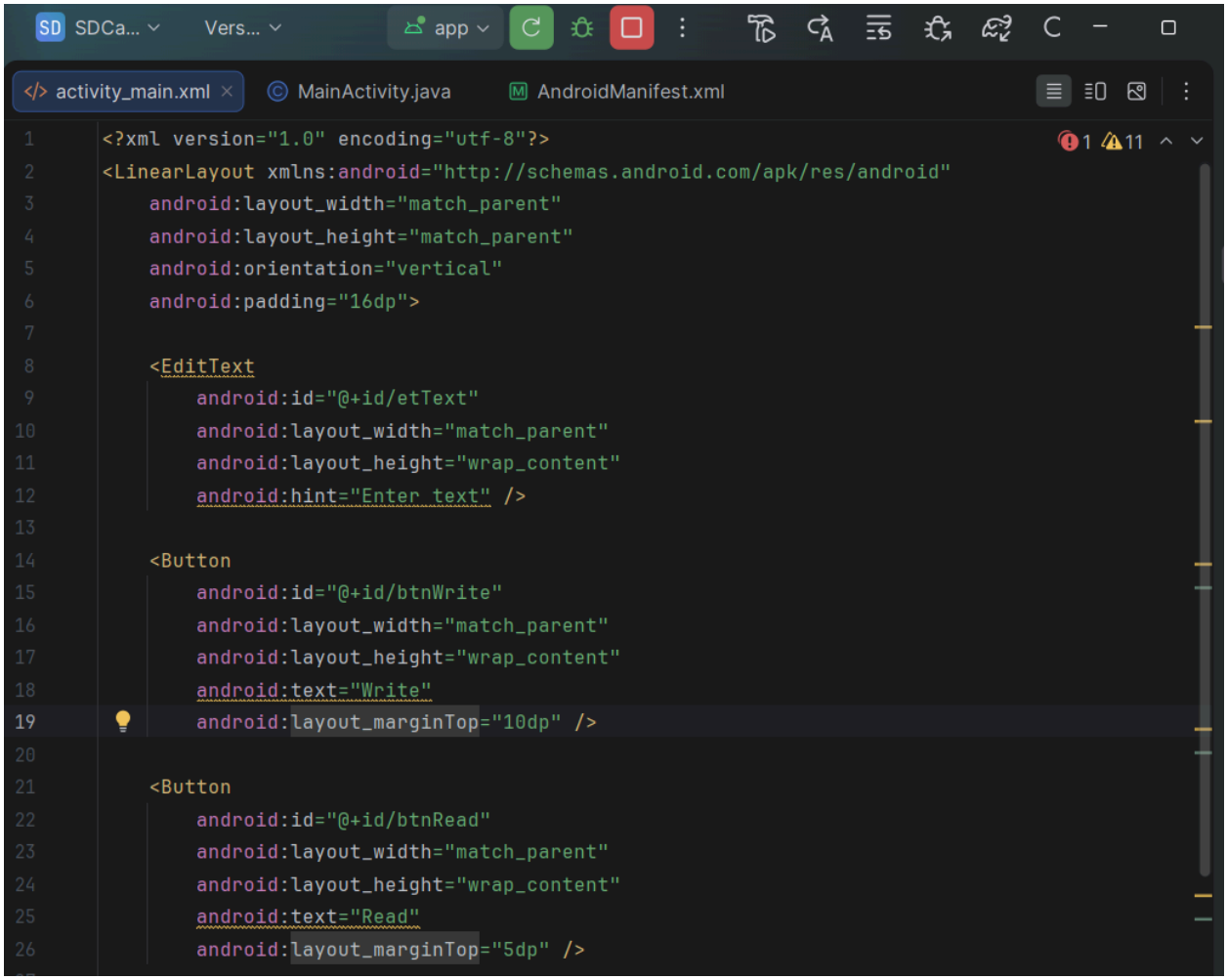
Program:

→AndroidManifest.xml



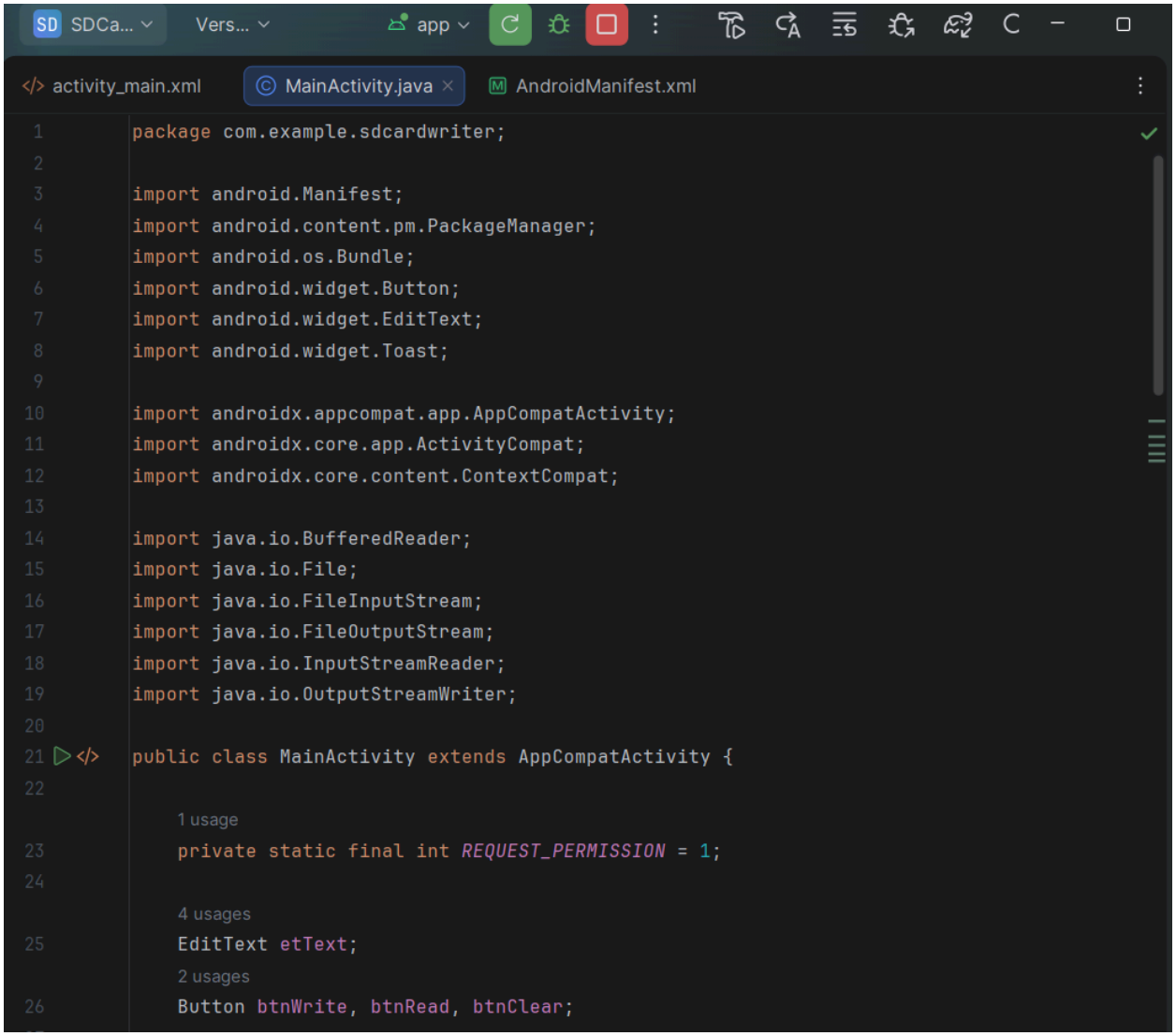
→activity_main.xml



```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter text" />

    <Button
        android:id="@+id/btnWrite"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Write"
        android:layout_marginTop="10dp" />

    <Button
        android:id="@+id/btnRead"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Read"
        android:layout_marginTop="5dp" />
```

```xml
    <Button
        android:id="@+id/btnClear"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Clear"
        android:layout_marginTop="5dp" />

</LinearLayout>
```

→MainActivity.java

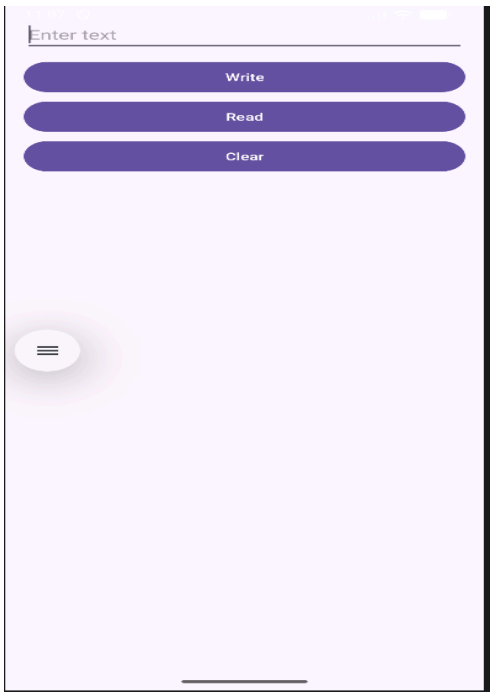


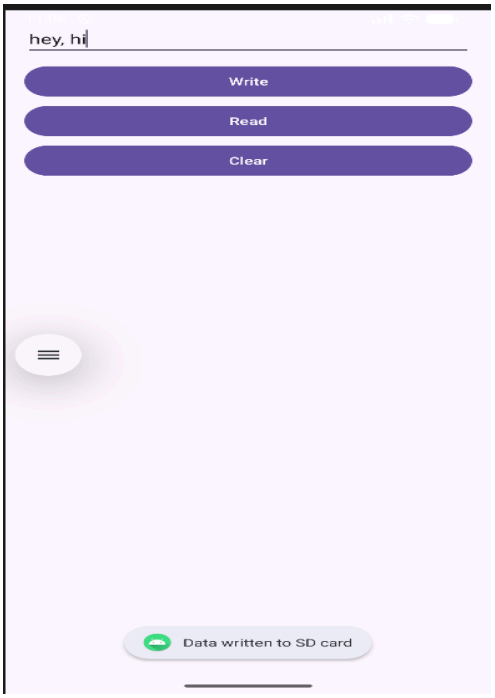
```java

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etText = findViewById(R.id.etText);
        btnWrite = findViewById(R.id.btnWrite);
        btnRead = findViewById(R.id.btnRead);
        btnClear = findViewById(R.id.btnClear);

        // Runtime permission (API 23+)
        if (ContextCompat.checkSelfPermission( context: this,
                Manifest.permission.WRITE_EXTERNAL_STORAGE)
                != PackageManager.PERMISSION_GRANTED) {

            ActivityCompat.requestPermissions( activity: this,
                    new String[]{Manifest.permission.WRITE_EXTERNAL_STORAGE,
                            Manifest.permission.READ_EXTERNAL_STORAGE},
                    REQUEST_PERMISSION);
        }

        btnWrite.setOnClickListener( View v -> writeToFile());
        btnRead.setOnClickListener( View v -> readFromFile());
        btnClear.setOnClickListener( View v -> etText.setText(""));
    }

    1 usage
```

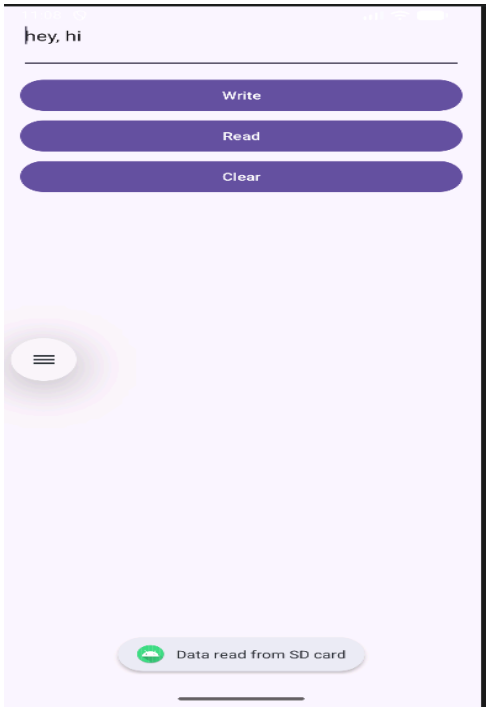
```
1 usage
private void writeToFile() {
    String text = etText.getText().toString();

    if (text.isEmpty()) {
        Toast.makeText( context: this,  text: "Enter text first", Toast.LENGTH_SHORT).show();
        return;
    }

    try {
        File file = new File(getExternalFilesDir( type: null),  child: "sdcard_data.txt");

        FileOutputStream fOut = new FileOutputStream(file);
        OutputStreamWriter writer = new OutputStreamWriter(fOut);

        writer.write(text);
        writer.close();
        fOut.close();

        Toast.makeText( context: this,  text: "Data written to SD card", Toast.LENGTH_SHORT).show();

    } catch (Exception e) {
        Toast.makeText( context: this,  text: "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
    }
}
```

```
1 usage
private void readFromFile() {
    try {
        File file = new File(getExternalFilesDir( type: null),  child: "sdcard_data.txt");

        FileInputStream fIn = new FileInputStream(file);
        BufferedReader reader = new BufferedReader(new InputStreamReader(fIn));

        StringBuilder buffer = new StringBuilder();
        String line;

        while ((line = reader.readLine()) != null) {
            buffer.append(line).append("\n");
        }

        reader.close();

        etText.setText(buffer.toString());
        Toast.makeText( context: this,  text: "Data read from SD card", Toast.LENGTH_SHORT).show();

    } catch (Exception e) {
        Toast.makeText( context: this,  text: "Error reading file: " + e.getMessage(), Toast.LENGTH_SHOR
    }
}
}
```

OUTPUT:


Enter text
Write
Read
Clear


hey, hi
Write
Read
Clear
Data written to SD card


hey, hi
Write
Read
Clear
Data read from SD card